

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Correlation and Pattern Analysis

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO4	Assessment:	Tool 3– Correlation and Pattern Analysis
Weightage:	25%	Total Marks:	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Dharshini H	Reg. No.:	192324268
Section Batch: /	B.Tech AI and DS	Date of Submission:	19-08-2026
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H , 192324268(Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

1. Correlation Analysis of Student Performance

Import student datasets containing **attendance**, **assignment marks**, **internal marks**, **study hours**, and **final marks**. Explore the tables, join relevant datasets, calculate correlations between variables, identify outliers, and use appropriate charts to explain the major patterns in student performance.

ANSWER:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 # -----
5 # 1. CREATE STUDENT DATASET
6 # -----
7 np.random.seed(42)
8 n = 500
9 data = pd.DataFrame({
10     "Student_ID": range(1, n + 1),
11     "Attendance": np.random.randint(50, 101, n),
12     "Assignment_Marks": np.random.randint(40, 101, n),
13     "Internal_Marks": np.random.randint(35, 101, n),
14     "Study_Hours": np.round(np.random.uniform(1, 10, n), 1),
15     "Department": np.random.choice(
16         ["AI&DS", "CSE", "ECE", "EEE", "IT"], n
17     ),
18     "Semester": np.random.choice(
19         [1, 2, 3, 4, 5, 6, 7, 8], n
20     )
21 })
22 # Create Final Marks based on other variables
23 data["Final_Marks"] = (
24     0.25 * data["Attendance"] +
25     0.25 * data["Assignment_Marks"] +
26     0.25 * data["Internal_Marks"] +
27     2.5 * data["Study_Hours"] +
28     np.random.normal(0, 5, n)
29 )
30 # Keep marks between 0 and 100
31 data["Final_Marks"] = data["Final_Marks"].clip(0, 100).round(2)
32 print("Dataset created successfully!")
33 print(data.head())
34 # -----
35 # 2. EXPLORE THE TABLE
36 # -----
37 print("\nDataset Shape:")
38 print(data.shape)
39 print("\nColumn Names:")
40 print(data.columns)
41 print("\nDataset Information:")
42 print(data.info())
43 print("\nStatistical Summary:")
44 print(data.describe())
45 print("\nMissing Values:")
46 print(data.isnull().sum())
47 # -----
48 # 3. SORTING
49 # -----
50 print("\nTop 10 Students:")
51 print(
52     data.sort_values(
53         by="Final_Marks",
54         ascending=False
55     ).head(10)
56 )
57 # -----
58 # 4. FILTERING
59 # -----
60 high_performers = data[data["Final_Marks"] >= 80]
61 print("\nStudents with Final Marks >= 80:")
62 print(high_performers.head())
63 good_attendance = data[data["Attendance"] >= 75]
64 print("\nStudents with Attendance >= 75:")
65 print(good_attendance.head())
66 # -----
67 # 5. GROUPING
68 # -----
69 department_performance = data.groupby(
70     ["Department"]
71 )[["Final_Marks"].mean().sort_values(ascending=False)]
72 print("\nAverage Final Marks by Department:")
73 print(department_performance)
74 semester_performance = data.groupby(
75     ["Semester"]
76 )[["Final_Marks"].mean()]
77 print("\nAverage Final Marks by Semester:")
78 print(semester_performance)
79 # -----
80 # 6. CORRELATION ANALYSIS
81 # -----
82 numeric_columns = [
83     "Attendance",
84     "Assignment_Marks",
85     "Internal_Marks",
86     "Study_Hours",
87     "Final_Marks"
88 ]
89 correlation = data[numeric_columns].corr()
90 print("\nCorrelation Matrix:")
91 print(correlation)
92 print("\nCorrelation with Final Marks:")
93 print(
94     correlation["Final_Marks"]
95     .sort_values(ascending=False)
96 )
```

```

98 # 7. CORRELATION HEATMAP
99 # -----
100 plt.figure(figsize=(9, 6))
101 plt.imshow(correlation,cmap="coolwarm",interpolation="nearest")
102 plt.colorbar()
103 plt.xticks(
104     range(len(correlation.columns)),
105     correlation.columns,
106     rotation=45)
107 plt.yticks(
108     range(len(correlation.columns)),
109     correlation.columns
110 )
111 plt.title("Correlation Matrix of Student Performance")
112 plt.tight_layout()
113 plt.show()
114 # -----
115 # 8. OUTLIER DETECTION USING IQR
116 # -----
117 def find_outliers(column):
118     Q1 = data[column].quantile(0.25)
119     Q3 = data[column].quantile(0.75)
120     IQR = Q3 - Q1
121     lower_limit = Q1 - 1.5 * IQR
122     upper_limit = Q3 + 1.5 * IQR
123     outliers = data[
124         (data[column] < lower_limit) |
125         (data[column] > upper_limit)
126     ]
127     return outliers
128 print("\nOutliers in Study Hours:")
129 print(find_outliers("Study_Hours"))
130 print("\nOutliers in Final Marks:")
131 print(find_outliers("Final_Marks"))
132 # -----
133 # 9. SCATTER PLOT
134 # -----
135 plt.figure(figsize=(8, 5))
136 plt.scatter(
137     data["Study_Hours"],
138     data["Final_Marks"],
139     alpha=0.6
140 )
141 plt.xlabel("Study Hours")
142 plt.ylabel("Final Marks")
143 plt.title("Study Hours vs Final Marks")
144 plt.grid(True)
145 plt.show()
146 # -----
147 # 10. ATTENDANCE VS FINAL MARKS
148 # -----
149 plt.figure(figsize=(8, 5))
150 plt.scatter(
151     data["Attendance"],
152     data["Final_Marks"],
153     alpha=0.6
154 )
155 plt.xlabel("Attendance (%)")
156 plt.ylabel("Final Marks")
157 plt.title("Attendance vs Final Marks")
158 plt.grid(True)
159 plt.show()
160 # -----
161 # 11. ASSIGNMENT MARKS VS FINAL MARKS
162 # -----
163 plt.figure(figsize=(8, 5))
164 plt.scatter(
165     data["Assignment_Marks"],
166     data["Final_Marks"],
167     alpha=0.6
168 )
169 plt.xlabel("Assignment Marks")
170 plt.ylabel("Final Marks")
171 plt.title("Assignment Marks vs Final Marks")
172 plt.grid(True)
173 plt.show()
174 # -----
175 # 12. INTERNAL MARKS VS FINAL MARKS
176 # -----
177 plt.figure(figsize=(8, 5))
178 plt.scatter(
179     data["Internal_Marks"],
180     data["Final_Marks"],
181     alpha=0.6
182 )
183 plt.xlabel("Internal Marks")
184 plt.ylabel("Final Marks")
185 plt.title("Internal Marks vs Final Marks")
186 plt.grid(True)
187 plt.show()
188 # -----
189 # 13. DEPARTMENT-WISE PERFORMANCE
190 # -----
191 plt.figure(figsize=(8, 5))
192 department_performance.plot(
193     kind="bar"
194 )
195 plt.xlabel("Department")
196 plt.ylabel("Average Final Marks")
197 plt.title("Average Final Marks by Department")
198 plt.xticks(rotation=0)
199 plt.tight_layout()
200 plt.show()
201 # -----
202 # 14. SEMESTER-WISE PERFORMANCE
203 # -----
204 plt.figure(figsize=(8, 5))
205 semester_performance.plot(
206     kind="line",
207     marker="o"
208 )
209 plt.xlabel("Semester")
210 plt.ylabel("Average Final Marks")
211 plt.title("Average Final Marks by Semester")
212 plt.grid(True)
213 plt.tight_layout()
214 plt.show()

```

```

215 # -----
216 # 15. TOP 10 STUDENTS
217 # -----
218 top10 = data.sort_values(
219     "Final_Marks",
220     ascending=False
221 ).head(10)
222 plt.figure(figsize=(10, 5))
223 plt.bar(
224     top10["Student_ID"].astype(str),
225     top10["Final_Marks"]
226 )
227 plt.xlabel("Student ID")
228 plt.ylabel("Final Marks")
229 plt.title("Top 10 Students by Final Marks")
230 plt.tight_layout()
231 plt.show()
232 # -----
233 # 16. FINAL RESULT
234 # -----
235 print("\n===== FINAL ANALYSIS =====")
236 print(
237     "Average Final Marks:",
238     round(data["Final_Marks"].mean(), 2)
239 )
240 print(
241     "Average Attendance:",
242     round(data["Attendance"].mean(), 2)
243 )
244 print(
245     "Average Study Hours:",
246     round(data["Study_Hours"].mean(), 2)
247 )
248 print(
249     "\nStrongest correlations with Final Marks:"
250     correlation["Final_Marks"]
251     .sort_values(ascending=False)

```

OUTPUT:

```

PS C:\Users\dhars\Documents\Query processing\lab_questions> & C:\Users\dhars\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dhars/Documents/Query processing/lab_questi
/CO4/data_2/at3_1.py"
Dataset created successfully!
  Student_ID  Attendance  Assignment_Marks  Internal_Marks  Study_Hours  Department  Semester  Final_Marks
0          1           88                67              61           8.9         AI&OS           8          77.23
1          2           78                44              66           9.9         CSE           7          73.33
2          3           64                86              84           7.9         ECE           3          82.69
3          4           92                88              95           4.8         IT            7          78.12
4          5           57                69              85           4.8         EEE           6          68.05

Dataset Shape:
(500, 8)

Column Names:
Index(['Student_ID', 'Attendance', 'Assignment_Marks', 'Internal_Marks',
      'Study_Hours', 'Department', 'Semester', 'Final_Marks'],
      dtype='str')

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Student_ID            500 non-null    int64
1   Attendance             500 non-null    int32
2   Assignment_Marks       500 non-null    int32
3   Internal_Marks         500 non-null    int32
4   Study_Hours            500 non-null    float64
5   Department             500 non-null    str
6   Semester               500 non-null    int64
7   Final_Marks            500 non-null    float64
dtypes: float64(2), int32(3), int64(2), str(1)
memory usage: 27.1 KB
None

```

```

Statistical Summary:
Student_ID Attendance Assignment_Marks Internal_Marks Study_Hours Semester Final_Marks
count 500.000000 500.000000 500.000000 500.000000 500.000000 500.000000 500.000000
mean 250.500000 75.406000 70.154000 66.914000 5.589200 4.490000 66.832480
std 144.481833 14.608353 17.746102 18.956299 2.643456 2.276019 10.943903
min 1.000000 50.000000 40.000000 35.000000 1.000000 1.000000 39.520000
25% 125.750000 64.000000 55.000000 51.000000 3.175000 3.000000 59.625000
50% 250.500000 76.000000 70.000000 67.000000 5.700000 4.000000 67.090000
75% 375.250000 88.000000 86.000000 83.000000 7.825000 6.000000 74.055000
max 500.000000 100.000000 100.000000 100.000000 10.000000 8.000000 97.780000

Missing Values:
Student_ID 0
Attendance 0
Assignment_Marks 0
Internal_Marks 0
Study_Hours 0
Department 0
Semester 0
Final_Marks 0
dtype: int64

Top 10 Students:
Student_ID Attendance Assignment_Marks Internal_Marks Study_Hours Department Semester Final_Marks
264 265 90 86 88 9.8 EEE 3 97.78
31 32 91 97 81 6.8 IT 3 97.69
188 189 88 97 100 9.3 IT 2 93.47
176 177 91 97 98 8.8 EEE 6 92.40
90 91 96 97 96 7.4 CSE 8 91.47
404 405 89 97 93 10.0 AI&DS 8 90.69
201 202 82 97 63 8.2 CSE 4 89.85
484 485 94 62 100 9.7 AI&DS 6 89.49
86 87 70 90 100 8.2 IT 4 89.36
221 222 98 99 94 7.3 EEE 4 89.32

Students with Final Marks >= 80:
Student_ID Attendance Assignment_Marks Internal_Marks Study_Hours Department Semester Final_Marks
2 3 64 86 84 7.9 ECE 3 82.69
20 21 79 97 60 8.1 EEE 5 82.92
30 31 76 58 86 9.5 AI&DS 5 82.02
31 32 91 97 81 6.8 IT 3 97.69
38 39 52 55 93 8.0 CSE 3 80.53

Students with Attendance >= 75:
Student_ID Attendance Assignment_Marks Internal_Marks Study_Hours Department Semester Final_Marks
0 1 88 67 61 8.9 AI&DS 8 77.23
1 2 78 44 66 9.9 CSE 7 73.33
3 4 92 88 95 4.8 IT 7 78.12
6 7 88 91 55 3.1 ECE 6 67.95
12 13 85 87 35 1.4 ECE 8 47.22

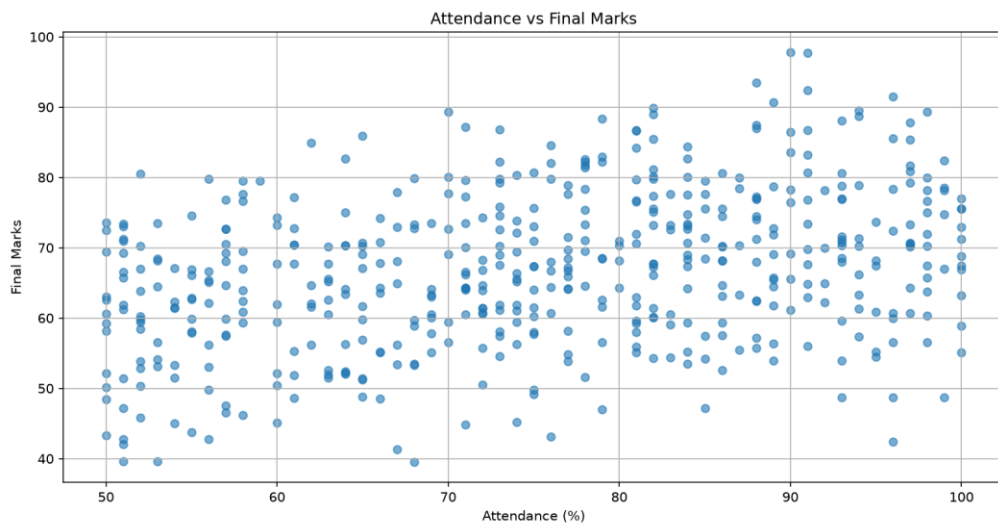
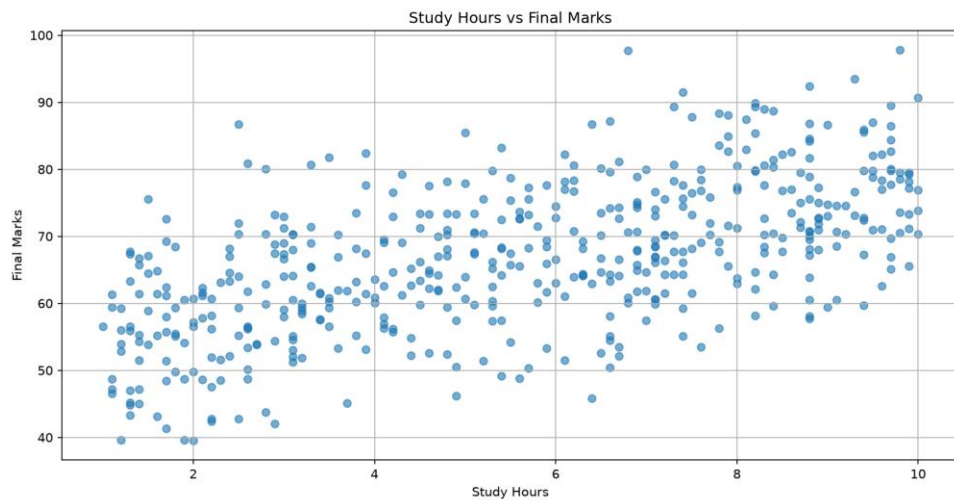
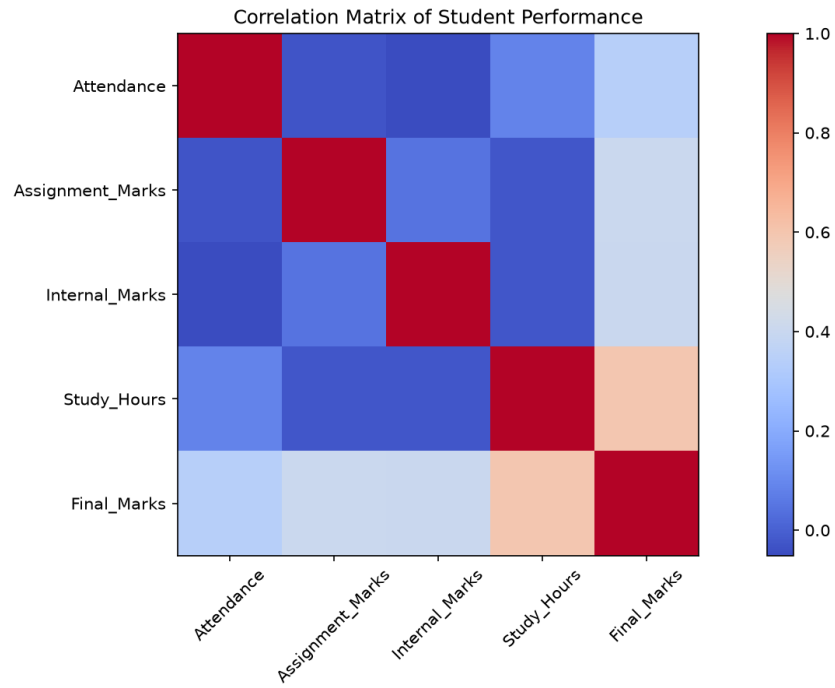
Average Final Marks by Department:
Department
ECE 67.794235
IT 67.443679
EEE 66.877544
AI&DS 66.618957
CSE 65.536931
Name: Final_Marks, dtype: float64

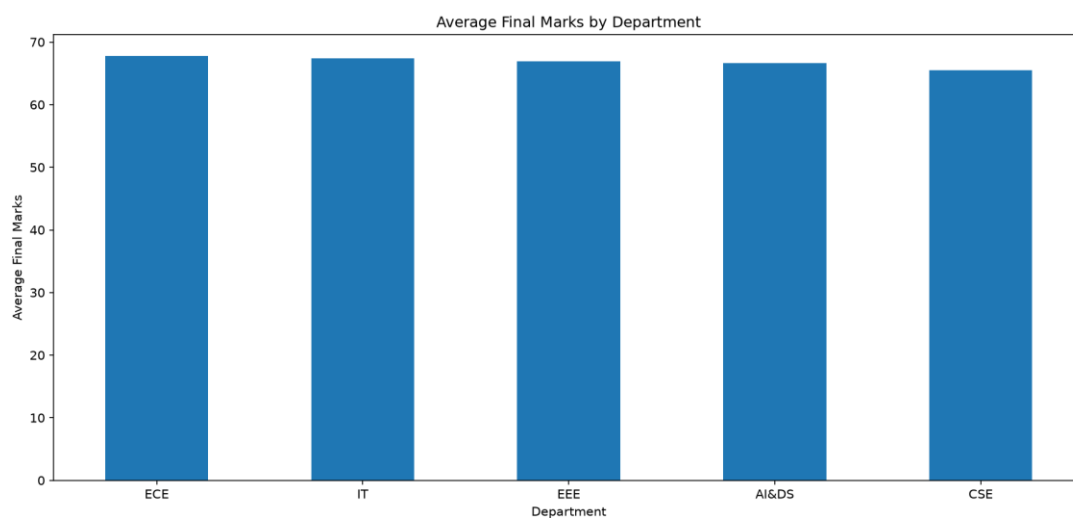
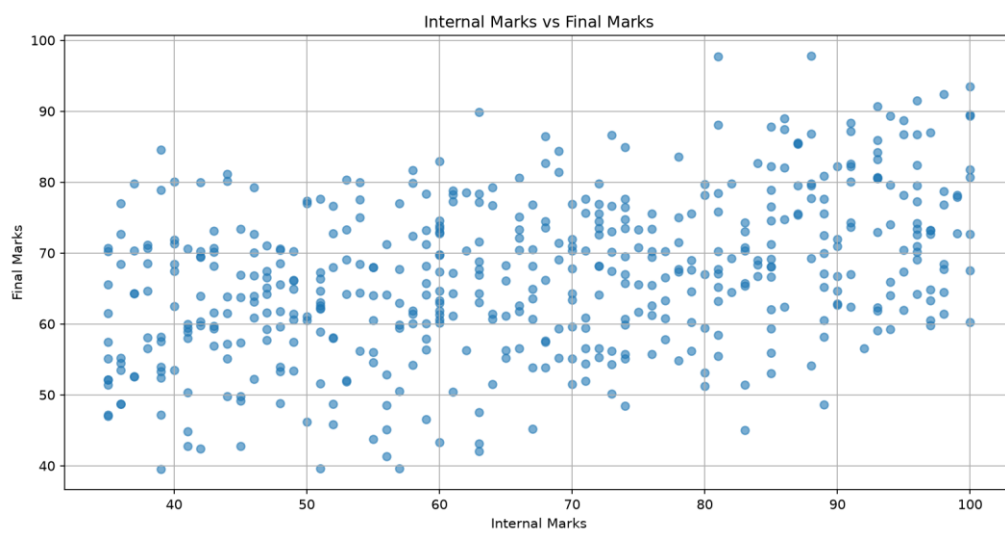
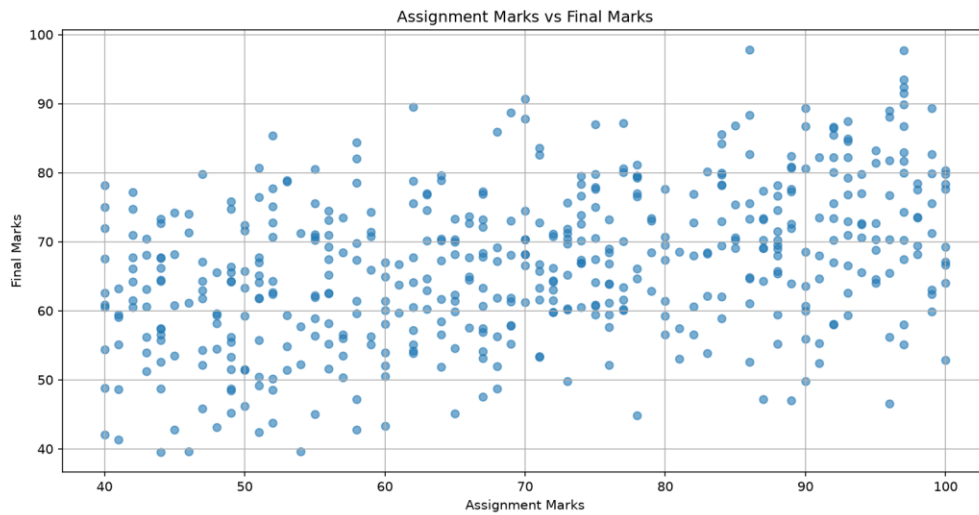
Average Final Marks by Semester:
Semester
1 66.060492
2 67.931311
3 68.557969
4 66.028732
5 67.230727
6 69.739265
7 63.712456
8 65.006667
Name: Final_Marks, dtype: float64

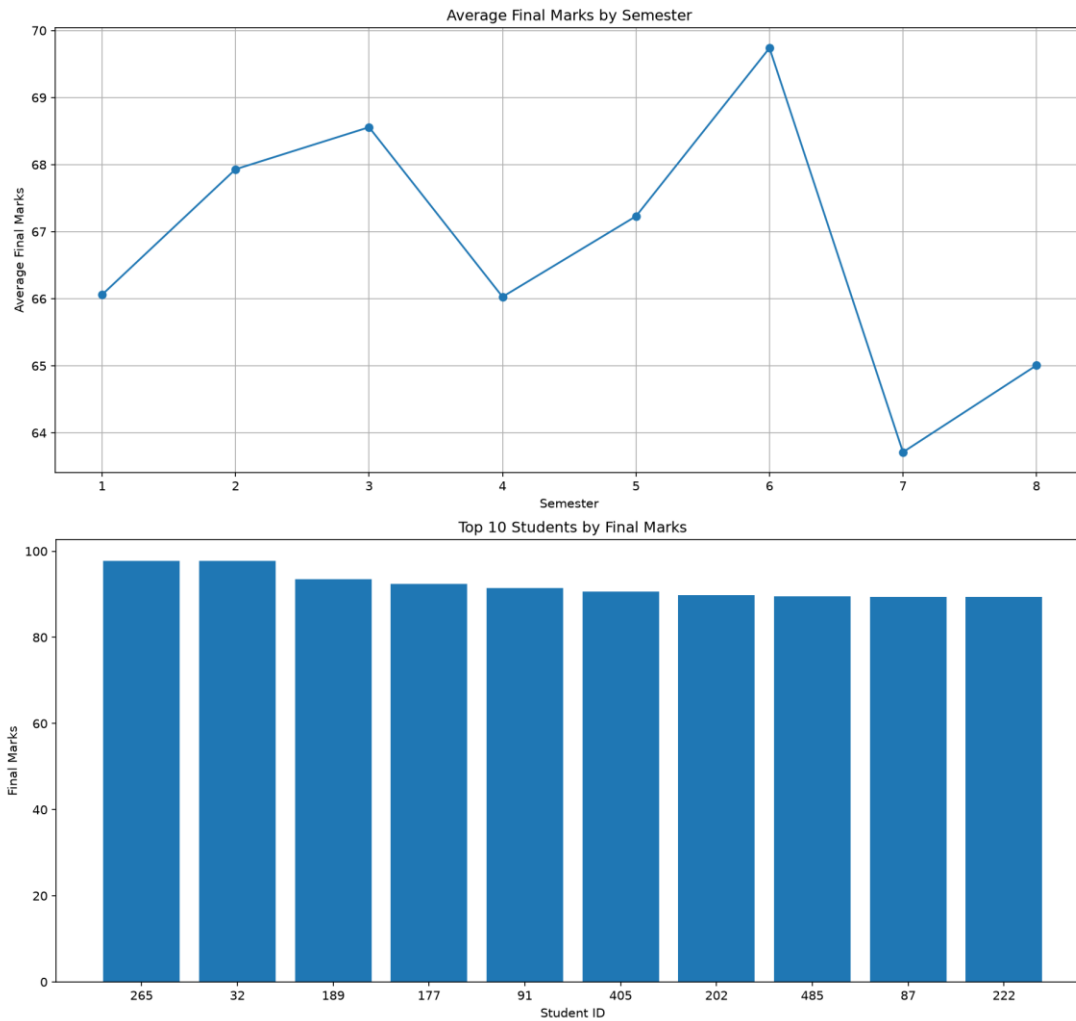
Correlation Matrix:
Attendance Assignment_Marks Internal_Marks Study_Hours Final_Marks
Attendance 1.000000 -0.026872 -0.050813 0.087137 0.342153
Assignment_Marks -0.026872 1.000000 0.044951 -0.025737 0.405756
Internal_Marks -0.050813 0.044951 1.000000 -0.022926 0.403984
Study_Hours 0.087137 -0.025737 -0.022926 1.000000 0.594711
Final_Marks 0.342153 0.405756 0.403984 0.594711 1.000000

Correlation with Final Marks:
Final_Marks 1.000000
Study_Hours 0.594711
Assignment_Marks 0.405756
Internal_Marks 0.403984
Attendance 0.342153
Name: Final_Marks, dtype: float64

```







2. Sales and Customer Pattern Analysis

Import sales and customer datasets and combine them using suitable table-joining techniques. Analyze the correlation between **customer age**, **purchase frequency**, **discount**, and **purchase amount**. Create meaningful customer groups, identify unusual purchasing patterns, and present the results using suitable visualizations.

ANSWER:


```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 # =====
5 # 1. CREATE CUSTOMER DATASET
6 # =====
7 np.random.seed(42)
8 n_customers = 500
9 customers = pd.DataFrame({"Customer_ID": range(1, n_customers + 1), "Age": np.random.randint(18, 65, n_customers), "Gender": np.random.choice(
10     ["Male", "Female"], n_customers),
11     "City": np.random.choice(["Chennai", "Bangalore", "Mumbai", "Delhi", "Hyderabad"], n_customers),
12     "Customer_Type": np.random.choice(["New", "Regular", "Premium"], n_customers, p=[0.30, 0.50, 0.20])})
13 customers.to_csv("customers.csv", index=False)
14 # =====
15 # 2. CREATE SALES DATASET
16 # =====
17 n_sales = 1500
18 sales = pd.DataFrame({
19     "Sale_ID": range(1, n_sales + 1),
20     "Customer_ID": np.random.randint(
21         1, n_customers + 1, n_sales),
22     "Purchase_Frequency": np.random.randint(
23         1, 20, n_sales),
24     "Discount": np.random.randint(
25         0, 51, n_sales),
26     "Purchase_Amount": np.random.randint(
27         200, 10001, n_sales),
28     "Product_Category": np.random.choice(
29         ["Electronics", "Clothing", "Grocery",
30          "Beauty", "Home"],
31         n_sales))
32 sales.to_csv("sales.csv", index=False)
33 print("customers.csv created")
34 print("sales.csv created")
35 # =====
36 # 3. IMPORT DATASETS
37 # =====
38 customers = pd.read_csv("customers.csv")
39 sales = pd.read_csv("sales.csv")
40
41 print("\nCUSTOMER DATA")
42 print(customers.head())
43
44 print("\nSALES DATA")
45 print(sales.head())
46
47 # =====
48 # 4. EXPLORE TABLES
49 # =====
50
51 print("\nCustomer Dataset Shape:")
52 print(customers.shape)
53
54 print("\nSales Dataset Shape:")
55 print(sales.shape)
56
57 print("\nCustomer Information:")
58 print(customers.info())
59
60 print("\nSales Information:")
61 print(sales.info())
62
63 print("\nCustomer Statistics:")
64 print(customers.describe())
65
66 print("\nSales Statistics:")
67 print(sales.describe())
68
69 # =====
70 # 5. CHECK MISSING VALUES
71 # =====
72
73 print("\nMissing Values in Customers:")
74 print(customers.isnull().sum())
75
76 print("\nMissing Values in Sales:")
77 print(sales.isnull().sum())
78
79 # =====
80 # 6. JOIN CUSTOMER AND SALES TABLES
81 # =====
82
83 merged_data = pd.merge(
84     sales,
85     customers,
86     on="Customer_ID",
87     how="inner"
88 )
89
90 print("\nMERGED DATASET")
91 print(merged_data.head())
92
93 print("\nMerged Dataset Shape:")
94 print(merged_data.shape)
95
96 # Save merged dataset
97 merged_data.to_csv(
98     "customer_sales_merged.csv",
99     index=False
100 )

```

```

109 # =====
110 # 7. SORTING
111 # =====
112 print("\nTop 10 Customers by Purchase Amount:")
113
114 top_customers = merged_data.sort_values(
115     by="Purchase_Amount",
116     ascending=False
117 ).head(10)
118
119 print(top_customers[
120     [
121         "Customer_ID",
122         "Age",
123         "Purchase_Frequency",
124         "Discount",
125         "Purchase_Amount"
126     ]
127 ])
128 # =====
129 # 8. FILTERING
130 # =====
131
132 high_value = merged_data[
133     merged_data["Purchase_Amount"] >= 8000
134 ]
135
136 print("\nHigh Value Purchases:")
137 print(high_value.head())
138
139 high_discount = merged_data[
140     merged_data["Discount"] >= 30
141 ]
142 print("\nPurchases with Discount >= 30%:")
143 print(high_discount.head())
144 # =====
145 # 9. CORRELATION ANALYSIS
146 # =====
147
148 numeric_columns = ["Age", "Purchase_Frequency", "Discount", "Purchase_Amount"]
149 correlation = merged_data[
150     numeric_columns].corr()
151 print("\nCORRELATION MATRIX")
152 print(correlation)
153 print("\nCorrelation with Purchase Amount:")
154 print(
155     correlation["Purchase_Amount"]
156     .sort_values(ascending=False)
157 )
158 # =====
159 # 10. CORRELATION HEATMAP
160 # =====
161 plt.figure(figsize=(8, 6))
162 plt.imshow(
163     correlation,
164     cmap="coolwarm",
165     interpolation="nearest")
166 plt.colorbar()
167 plt.xticks(
168     range(len(correlation.columns)),
169     correlation.columns,
170     rotation=45)
171 plt.yticks(
172     range(len(correlation.columns)),
173     correlation.columns)
174 plt.title("Correlation Between Customer and Sales Variables")
175 plt.tight_layout()
176 plt.show()
177 # =====
178 # 11. CUSTOMER GROUPING
179 # =====
180
181
182 customer_groups = merged_data.groupby(
183     "Customer_Type"
184 ).agg(
185     Average_Purchase=("Purchase_Amount", "mean"),
186     Average_Discount=("Discount", "mean"),
187     Average_Frequency=("Purchase_Frequency", "mean"),
188     Number_of_Purchases=("Purchase_Amount", "count")
189 )
190
191 print("\nCUSTOMER GROUP ANALYSIS")
192 print(customer_groups)
193
194 # =====
195 # 12. CITY-WISE SALES
196 # =====
197
198
199 city_sales = merged_data.groupby(
200     "city"
201 ).["Purchase_Amount"].sum().sort_values(
202     ascending=False
203 )
204
205 print("\nCITY-WISE SALES")
206 print(city_sales)
207
208

```

```

210 # =====
211 # 13. PRODUCT CATEGORY ANALYSIS
212 # =====
213
214 category_sales = merged_data.groupby(
215     ["Product_Category"]
216 )["Purchase_Amount"].sum().sort_values(
217     ascending=False
218 )
219
220 print("\nPRODUCT CATEGORY SALES")
221 print(category_sales)
222
223 # =====
224 # 14. BAR CHART - CUSTOMER GROUPS
225 # =====
226
227 plt.figure(figsize=(8, 5))
228
229 customer_groups["Average_Purchase"].plot(
230     kind="bar"
231 )
232
233 plt.xlabel("Customer Type")
234 plt.ylabel("Average Purchase Amount")
235 plt.title("Average Purchase Amount by Customer Type")
236
237 plt.xticks(rotation=0)
238 plt.tight_layout()
239 plt.show()
240
241 # =====
242 # 15. BAR CHART - CITY SALES
243 # =====
244
245 plt.figure(figsize=(8, 5))
246
247 city_sales.plot(
248     kind="bar"
249 )
250
251 plt.xlabel("City")
252 plt.ylabel("Total Purchase Amount")
253 plt.title("Total Sales by City")
254
255 plt.xticks(rotation=45)
256 plt.tight_layout()
257 plt.show()
258
259 # =====
260 # 16. BAR CHART - PRODUCT CATEGORY
261 # =====
262
263 plt.figure(figsize=(8, 5))
264
265 category_sales.plot(
266     kind="bar"
267 )
268
269 plt.xlabel("Product Category")
270 plt.ylabel("Total Purchase Amount")
271 plt.title("Sales by Product Category")
272
273 plt.xticks(rotation=45)
274 plt.tight_layout()
275 plt.show()
276
277 # =====
278 # 17. AGE VS PURCHASE AMOUNT
279 # =====
280
281 plt.figure(figsize=(8, 5))
282
283 plt.scatter(
284     merged_data["Age"],
285     merged_data["Purchase_Amount"],
286     alpha=0.5
287 )
288
289 plt.xlabel("Customer Age")
290 plt.ylabel("Purchase Amount")
291 plt.title("Customer Age vs Purchase Amount")
292 plt.grid(True)
293 plt.show()
294
295 # =====
296 # 18. PURCHASE FREQUENCY VS PURCHASE AMOUNT
297 # =====
298
299 plt.figure(figsize=(8, 5))
300
301 plt.scatter(
302     merged_data["Purchase_Frequency"],
303     merged_data["Purchase_Amount"],
304     alpha=0.5
305 )
306
307 plt.xlabel("Purchase Frequency")
308 plt.ylabel("Purchase Amount")
309 plt.title("Purchase Frequency vs Purchase Amount")
310 plt.grid(True)
311 plt.show()

```

```

310 # =====
311 # 19. DISCOUNT VS PURCHASE AMOUNT
312 # =====
313 plt.figure(figsize=(8, 5))
314 plt.scatter(merged_data["Discount"],merged_data["Purchase_Amount"],alpha=0.5)
315 plt.xlabel("Discount (%)")
316 plt.ylabel("Purchase Amount")
317 plt.title("Discount vs Purchase Amount")
318 plt.grid(True)
319 plt.show()
320 # =====
321 # 20. OUTLIER DETECTION USING IQR
322 # =====
323 def detect_outliers(df, column):
324     Q1 = df[column].quantile(0.25)
325     Q3 = df[column].quantile(0.75)
326     IQR = Q3 - Q1
327     lower_limit = Q1 - 1.5 * IQR
328     upper_limit = Q3 + 1.5 * IQR
329     outliers = df[
330         (df[column] < lower_limit) |
331         (df[column] > upper_limit)
332     ]
333     return outliers, lower_limit, upper_limit
334 amount_outliers, lower, upper = detect_outliers(merged_data,"Purchase_Amount")
335 print("\nPURCHASE AMOUNT OUTLIERS")
336 print(amount_outliers.head(20))
337 print("\nNumber of Purchase Amount Outliers:",len(amount_outliers))
338 frequency_outliers, lower, upper = detect_outliers(merged_data, "Purchase_Frequency")
339 print("\nPURCHASE FREQUENCY OUTLIERS")
340 print(frequency_outliers.head(20))
341 print("\nNumber of Purchase Frequency Outliers:",len(frequency_outliers)),
342 # =====
343 # 21. BOXPLOT FOR OUTLIERS
344 # =====
345 plt.figure(figsize=(8, 5))
346 plt.boxplot(merged_data["Purchase_Amount"])
347 plt.ylabel("Purchase Amount")
348 plt.title("Purchase Amount Outlier Detection")
349 plt.show()
350 # =====
351 # 22. UNUSUAL PURCHASING PATTERNS
352 # =====
353 unusual = merged_data[
354     (
355         merged_data["Purchase_Amount"] > 8000
356     )
357     &
358     (
359         merged_data["Purchase_Frequency"] <= 3
360     )
361 ]
362
363 print("\nUNUSUAL PURCHASING PATTERNS")
364 print(unusual.head(20))
365
366 print(
367     "\nNumber of unusual purchasing records:",
368     len(unusual)
369 )
370
371 # =====
372 # 23. PREMIUM CUSTOMER ANALYSIS
373 # =====
374
375 premium = merged_data[
376     merged_data["Customer_Type"] == "Premium"
377 ]
378
379 print("\nPREMIUM CUSTOMER ANALYSIS")
380
381 print(
382     "Average Premium Purchase:",
383     round(
384         premium["Purchase_Amount"].mean(),
385         2
386     )
387 )
388
389 print(
390     "Average Premium Discount:",
391     round(
392         premium["Discount"].mean(),
393         2
394     )
395 )
396
397 print(
398     "Average Premium Frequency:",
399     round(
400         premium["Purchase_Frequency"].mean(),
401         2
402     )
403 )
404
405

```

```

408 # 24. FINAL SUMMARY
409 # =====
410 print("\n=====")
411 print("          FINAL SALES ANALYSIS")
412 print("=====")
413 print(
414     "\nTotal Sales:",
415     round(
416         merged_data["Purchase_Amount"].sum(),
417         2))
418 print(
419     "Average Purchase Amount:",
420     round(
421         merged_data["Purchase_Amount"].mean(),
422         2))
423 print(
424     "Average Customer Age:",
425     round(
426         merged_data["Age"].mean(),
427         2))
428 print(
429     "Average Purchase Frequency:",
430     round(
431         merged_data["Purchase_Frequency"].mean(),
432         2))
433 print(
434     "Average Discount:",
435     round(
436         merged_data["Discount"].mean(),
437         2))
438 print("\nCorrelation with Purchase Amount:")
439 print(
440     correlation["Purchase_Amount"]
441     ,sort_values(ascending=False))
442 print("\nAnalysis completed successfully!")

```

OUTPUT:

```

CUSTOMER DATA
  Customer_ID  Age Gender      City Customer_Type
0           1   56   Male    Chennai      Regular
1           2   46   Male    Mumbai      Premium
2           3   32   Male  Hyderabad        New
3           4   60   Male    Chennai      Regular
4           5   25   Male  Bangalore        New

SALES DATA
  Sale_ID  Customer_ID  Purchase_Frequency  Discount  Purchase_Amount  Product_Category
0         1           1           124           1         2676           Grocery
1         2           2           399          13         4803           Electronics
2         3           3           281          17         3753           Grocery
3         4           4           292           3         420           Clothing
4         5           5           203           8         5018           Clothing

Customer Dataset Shape:
(500, 5)

Sales Dataset Shape:
(1500, 6)

Customer Information:
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Customer_ID  500 non-null    int64
1   Age          500 non-null    int64
2   Gender       500 non-null    str
3   City         500 non-null    str
4   Customer_Type 500 non-null    str
dtypes: int64(2), str(3)
memory usage: 28.4 KB
None

```

```

Sales Information:
<class 'pandas.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Sale_ID      1500 non-null  int64
1   Customer_ID  1500 non-null  int64
2   Purchase_Frequency  1500 non-null  int64
3   Discount     1500 non-null  int64
4   Purchase_Amount  1500 non-null  int64
5   Product_Category  1500 non-null  str
dtypes: int64(5), str(1)
memory usage: 81.1 KB
None

Customer Statistics:
      Customer_ID      Age
count  500.000000  500.000000
mean    250.500000    41.278000
std     144.481833    13.389072
min       1.000000    18.000000
25%     125.750000    30.000000
50%     250.500000    42.000000
75%     375.250000    52.000000
max      500.000000    64.000000

Sales Statistics:
      Sale_ID  Customer_ID  Purchase_Frequency  Discount  Purchase_Amount
count  1500.000000  1500.000000    1500.000000    1500.000000    1500.000000
mean     750.500000    250.503333     10.182667     25.187333    5045.952000
std     433.157015    144.945038      5.497040     14.549411    2824.110497
min       1.000000     1.000000      1.000000      0.000000     201.000000
25%     375.750000    126.000000      5.000000     13.000000    2672.000000
50%     750.500000    251.000000     10.000000     25.000000    4980.500000
75%    1125.250000    381.250000     15.000000     38.000000    7470.750000
max    1500.000000    500.000000     19.000000     50.000000    9999.000000

```

Missing Values in Customers:

```
Customer_ID    0
Age            0
Gender         0
City           0
Customer_Type  0
dtype: int64
```

Missing Values in Sales:

```
Sale_ID        0
Customer_ID     0
Purchase_Frequency  0
Discount        0
Purchase_Amount  0
Product_Category 0
dtype: int64
```

MERGED DATASET

	Sale_ID	Customer_ID	Purchase_Frequency	Discount	...	Age	Gender	City	Customer_Type
0	1	124	1	16	...	61	Female	Bangalore	Premium
1	2	399	13	14	...	49	Male	Bangalore	Premium
2	3	281	17	41	...	28	Male	Delhi	New
3	4	292	3	14	...	35	Female	Mumbai	Regular
4	5	203	8	14	...	20	Male	Mumbai	New

[5 rows x 10 columns]

Merged Dataset Shape:

(1500, 10)

Top 10 Customers by Purchase Amount:

	Customer_ID	Age	Purchase_Frequency	Discount	Purchase_Amount
1330	142	32	13	26	9999
1235	131	54	9	27	9999
1079	277	37	14	34	9999
190	381	42	9	48	9997
1402	86	41	8	37	9993
172	487	62	5	30	9975
1266	350	49	10	18	9974
1479	347	52	14	10	9949
1025	327	64	17	18	9944
349	117	29	13	43	9943

High Value Purchases:

	Sale_ID	Customer_ID	Purchase_Frequency	Discount	...	Age	Gender	City	Customer_Type
12	13	290	17	25	...	42	Male	Mumbai	Regular
15	16	452	8	17	...	20	Male	Bangalore	Premium
16	17	496	11	12	...	37	Male	Hyderabad	Premium
17	18	373	1	15	...	50	Female	Bangalore	Regular
28	29	420	19	38	...	47	Male	Delhi	Regular

[5 rows x 10 columns]

Purchases with Discount >= 30%:

	Sale_ID	Customer_ID	Purchase_Frequency	Discount	...	Age	Gender	City	Customer_Type
2	3	281	17	41	...	28	Male	Delhi	New
5	6	51	14	38	...	45	Female	Bangalore	New
8	9	277	13	31	...	37	Female	Bangalore	Regular
10	11	470	13	39	...	38	Female	Mumbai	Regular
20	21	27	10	43	...	39	Female	Chennai	Regular

[5 rows x 10 columns]

CORRELATION MATRIX

	Age	Purchase_Frequency	Discount	Purchase_Amount
Age	1.000000	0.018962	-0.007224	-0.001029
Purchase_Frequency	0.018962	1.000000	0.052221	0.013002
Discount	-0.007224	0.052221	1.000000	0.045947
Purchase_Amount	-0.001029	0.013002	0.045947	1.000000

Correlation with Purchase Amount:

```
Purchase_Amount    1.000000
Discount           0.045947
Purchase_Frequency 0.013002
Age               -0.001029
Name: Purchase_Amount, dtype: float64
[]
```

